Diffusion-Based Image Generation on MNIST

Yu-Lun Yeh (M132040013)

November 18, 2025

Abstract

This report studies a diffusion-based generative model for the MNIST dataset (28×28, RGB). We adopt a cosine noise schedule, a FiLM-modulated UNet with dynamic group normalization and scaled dot-product self-attention, a MinSNR-weighted quadratic loss, EMA averaging of parameters, and DDIM sampling. We generate 10,000 images and evaluate quality using Fréchet Inception Distance (FID). On our run, we obtain **FID = 17.63** comparing the generated images to the training set.

1 Introduction

Diffusion generative models define a forward *noising* process that gradually perturbs data to Gaussian noise and a learned reverse process that progressively removes noise. From a statistical perspective, the forward process specifies a family of conditional normal laws $\{q(x_t \mid x_0)\}$ with time-dependent signal-to-noise ratio (SNR), while the reverse model estimates the score (or, equivalently here, the noise) to recover a cleaner latent at earlier times.

2 Dataset and Preprocessing

Dataset. MNIST digits of size 28×28 with three channels (RGB representation). **Normalization.** We map pixel values to $[-1, 1]$ by $(x/255 - 0.5)/0.5$. No augmentation is used. A batch size of 128 and shuffled mini-batches are employed in training.

3 Method

3.1 Forward (Noising) Process

Let $\{\beta_t\}_{t=1}^{T}$ be a noise schedule with $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. The forward conditional distribution is

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t)\mathbf{I}\big), \tag{1}$$

with a cosine schedule for β_t. Equation (1) implies we can draw x_t in closed form given x_0 and an independent standard normal ε.

3.2 Denoiser and Objective

We train a neural predictor $\varepsilon_\theta(x_t, t)$ for the additive noise in (1). To stabilize learning across time, we use a MinSNR-γ weighting:

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}, \qquad w(t) = \frac{\min\{\mathrm{SNR}(t), \gamma\}}{\mathrm{SNR}(t)}, \tag{2}$$

$$\mathcal{L}(\theta) = \mathbb{E}\Big[w(t)\, \big\|\varepsilon - \varepsilon_\theta(x_t, t)\big\|_2^2\Big], \tag{3}$$

where $\gamma = 5$ in our experiments. We maintain an exponential moving average (EMA) of parameters with decay 0.999 for sampling.

3.3 Network Architecture

The denoiser is a UNet backbone with two down-sampling stages ($28 \to 14 \to 7$), a bottleneck, and two symmetric up-sampling stages. Time is embedded by a sinusoidal map followed by an MLP, then injected via FiLM (feature-wise linear modulation: per-channel scale/shift) into residual blocks. For normalization we use dynamic GroupNorm (choosing the number of groups that exactly divides the channel size), which behaves stably across widths. We place scaled dot-product multi-head self-attention at spatial sizes 14×14 and 7×7 to model long-range structure. Down-sampling uses stride-2 convolutions; up-sampling uses nearest-neighbor interpolation followed by convolution.

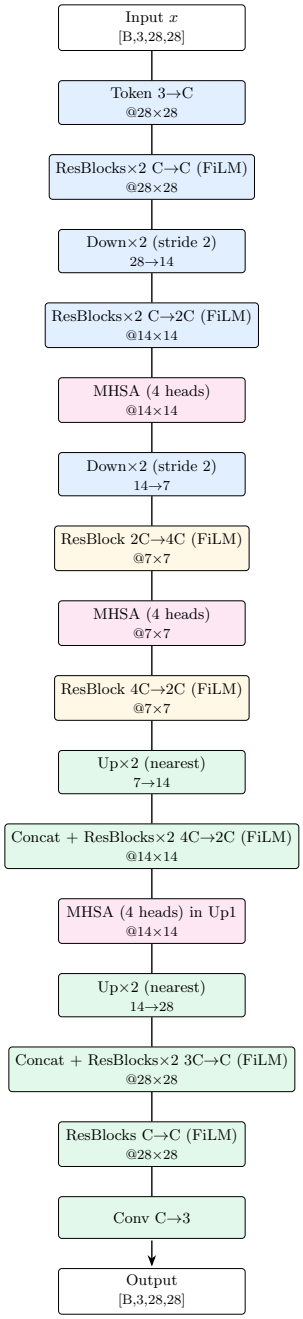

Figure 1: Simplified flowchart of *UNet_FiLM_Attn*. FiLM (sinusoidal+MLP time embedding) is applied inside each ResBlock; skip connections are omitted for clarity.

3.4 Sampling (DDIM, $\eta = 0$)

Given $\varepsilon_\theta(x_t, t)$, we form the denoised estimate

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{4}$$

Let $t' < t$ be an earlier step; the DDIM jump with $\eta = 0$ is

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}}\,\hat{x}_0 + \sqrt{1-\bar{\alpha}_{t'}}\,\varepsilon_\theta(x_t, t). \tag{5}$$

We use 50 evenly spaced steps from $T-1$ down to 0.

4 Training Setup

Item	Value
Total steps T	1000 (cosine schedule)
Optimizer	AdamW, learning rate 5×10^{-4}, weight decay 0
Batch size / Epochs	128 / 80
Gradient clip	1.0
Time embedding dim	256
Base channels	64
MinSNR γ	5
EMA decay	0.999
Sampler	DDIM, 50 steps
Generated images	10,000 PNGs
Hardware	Google Colab (CUDA)

Table 1: Training hyperparameters and environment.

5 Results

5.1 Diffusion Process Visualization

Per assignment, we divide T=1000 into 7 equal parts (record every 142 steps) and show 8 distinct samples, yielding a grid of 8×8 images.

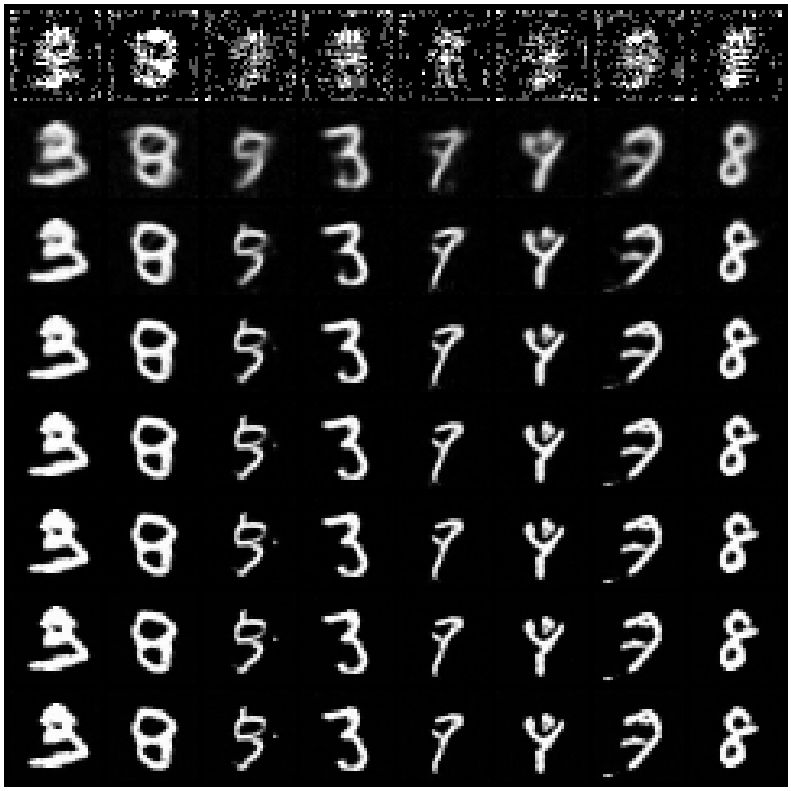

Figure 2: Diffusion process visualization: 8 samples × 8 snapshots.

5.2 Random Generated Samples

Provide randomly generated samples for qualitative inspection.

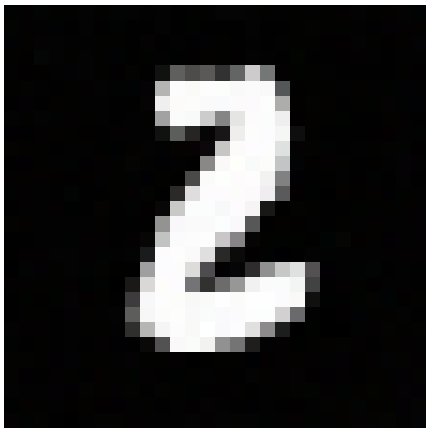

Figure 3: Randomly generated samples.

5.3 Quantitative Evaluation (FID)

We evaluate Fréchet Inception Distance between our 10k generated images and the training set using `pytorch-fid`. Our run yields **FID = 17.63**.

6 Discussion

From a statistical viewpoint, the MinSNR weighting balances contributions from early and late times by capping the effective SNR. EMA averaging reduces variance of the estimator

Setting	Images	FID ↓
Ours (EMA, DDIM-50)	10,000	**17.63**

Table 2: FID against MNIST training set. Lower is better.

at sampling time. Attention at 14×14 and 7×7 helps capture global strokes while FiLM modulation injects time-specific scale and shift to each residual block. Under our setting, these components collectively improve sample sharpness and FID.

Appendix A: Training and Sampling Pseudocode

Algorithm 1 Training (noise prediction with MinSNR weighting)

Initialize θ.
for each mini-batch x_0 **do**
    Sample $t \sim \mathrm{Unif}\{0, \ldots, T-1\}$ and $\varepsilon \sim \mathcal{N}(0, I)$.
    Form $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon$.
    Predict $\hat{\varepsilon} = \varepsilon_\theta(x_t, t)$.
    Compute $w(t) = \min\{\mathrm{SNR}(t), \gamma\}/\mathrm{SNR}(t)$.
    Update θ by descending $\nabla_\theta\, w(t) \,\|\, \varepsilon - \hat{\varepsilon} \,\|_2^2$.
    Update EMA parameters.
end for

Algorithm 2 DDIM Sampling ($\eta = 0$)

Initialize $x_{T-1} \sim \mathcal{N}(0, I)$.
for evenly spaced $t = T - 1 \to 0$ **do**
    $\hat{x}_0 = \big(x_t - \sqrt{1 - \bar{\alpha}_t}\, \varepsilon_\theta(x_t, t)\big) / \sqrt{\bar{\alpha}_t}$.
    Set $x_{t'} = \sqrt{\bar{\alpha}_{t'}}\, \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t'}}\, \varepsilon_\theta(x_t, t)$ where $t' < t$ is the next scheduled step.
end for
Return x_0.

References

[1] J. Ho, A. Jain, P. Abbeel. Denoising Diffusion Probabilistic Models. NeurIPS 2020.

[2] J. Song, C. Meng, S. Ermon. Denoising Diffusion Implicit Models. ICLR 2021.

[3] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, S. Hochreiter. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. NeurIPS 2017.